Phase 2: Backend Development & Configurations

Goal: build the data model and server-side automation powering every booking.

Data Architecture

Custom table `u_metro_ticket` (extends Task table):

Field	Type	Purpose
`u_ticket_number`	String	Unique ticket ID
`u_source_station`	Reference/Choice	Departure station
`u_destination_station`	Reference/Choice	Arrival station
`u_journey_date`	Date	Travel date
`u_passenger_count`	Integer	Passenger count
`u_fare_amount`	Currency	Calculated fare
`u_qr_code`	String/Image	Generated QR reference

Mandatory checks set at dictionary level as a backend safeguard.

Layered design:

```
UI Layer     → Service Portal + Catalog
Logic Layer  → Business Rules + Script Includes
Data Layer   → u_metro_ticket
Insight Layer→ Reports & Dashboards
```

Naming Conventions

- Tables/fields prefixed `u_`
- Script Includes named by function (`MetroFareCalculator`, `MetroQRGenerator`)
- Business Rules named by trigger + purpose (e.g. "Before Insert – Fare Calc")

Fare Calculation Logic

Script Include `MetroFareCalculator` — reusable, testable, callable from multiple rules:

```
var MetroFareCalculator = Class.create();
MetroFareCalculator.prototype = {
    calculateFare: function(source, destination, passengerC
ount) {
        return this._getRouteFare(source, destination) * pa
ssengerCount;
    },
    _getRouteFare: function(source, destination) {
        // lookup against station-pair fare mapping
    },
    type: 'MetroFareCalculator'
};
```

Business Rules

Rule	Trigger	Purpose
Fare Calculation	Before Insert	Populates `u_fare_amount`
Ticket Numbering	Before Insert	Assigns unique ticket number
QR Trigger	After Insert	Fires only on a fully valid, committed ticket

QR Code Generation

1. QR Trigger rule calls Script Include `MetroQRGenerator`
2. Ticket details packaged and sent to external QR API
3. Returned QR image/URL stored on `u_qr_code`

Automation flow:

```
Submit → Fare Calc (before insert) → Ticket Number (before insert)
→ Record Committed → QR Trigger (after insert) → QR API call
→ QR stored → Notification sent
```

Error Handling

- API timeout on QR call → logged, retried once, flagged for admin if still failing
- Invalid station pair → Script Include returns null fare, Business Rule blocks insert with a clear error

Notifications

- Scoped to `u_metro_ticket`, fires after QR is populated
- Includes ticket number, journey details, fare, QR image
- Recipient = requesting user

Backend Validation (Pre-UI)

- Manually inserted test record → confirmed fare, ticket number, QR, and email all worked independent of any UI

Deliverable: fully functioning, independently testable backend ready for Phase 3.